STOQUIFY / AGENT AND SKILL ASSURANCE

Stoquify Next Five Essential Skills - Expert Assessment and Recommendation

Five external skills ranked for governed Stoquify adoption

EXPERT REPORT

Prepared 2026-08-05

Evidence-led | read-only assessment | governance-first

Pinned sources. Explicit controls. Reproducible evidence.

Date: 2026-08-05 **Decision scope:** Five additional external skills, assessed without installation or execution **Research basis:** Stoquify repository evidence plus current upstream primary sources **Decision owner:** Stoquify engineering, security, product, and assurance leadership

Executive Decision

Stoquify should use these five skills as a governed engineering-assurance layer, not as autonomous authorities. The strongest candidate is Supabase's PostgreSQL best-practices skill because it directly addresses Stoquify's highest-risk technical gap: tenant isolation and database correctness. Addy Osmani's web-quality audit is also worth adopting as a lightweight review standard, provided its checklist is backed by deterministic Lighthouse, accessibility, and browser evidence.

Sentry's Next.js skill can create major operational value, but its example configuration is unsafe for Stoquify as written: it enables default PII transmission and server local-variable capture. It is a controlled, privacy-hardened pilot only. Trail of Bits' supply-chain auditor is useful for maintainer and takeover-risk triage, but it is not an SCA, SBOM, license, or active-vulnerability gate and cannot close Stoquify's release-assurance finding by itself. Anthropic's webapp-testing skill has the lowest incremental value because Stoquify already has a substantial TypeScript Playwright estate; its reconnaissance-before-action method should be adapted, while its Python server helper should not be installed as-is.

Ranked recommendation

Rank	Candidate	Score	Decision	Required boundary
1	Supabase `supabase-postgres-best-practices`	93.5	Adopt now	Governed, pinned adoption for database design and review
2	Addy Osmani `web-quality-audit`	83.5	Adopt now	Adopt as a checklist; require tool-generated evidence
3	Sentry `sentry-nextjs-sdk`	78.5	Controlled pilot	Staging-only, privacy-hardened configuration; never use examples unchanged
4	Trail of Bits `supply-chain-risk-auditor`	74.5	Controlled pilot	Advisory dependency-risk triage paired with real security gates
5	Anthropic `webapp-testing`	63.5	Selectively adapt	Reuse method inside existing Playwright architecture; do not install helper as-is

Portfolio recommendation: two governed adoptions, two narrow pilots, and one pattern-only adaptation. No candidate should receive production credentials, business-write authority, or independent certification authority.

Why These Five, Why Now

Stoquify's whole-system audit dated 2026-08-03 records an overall **NO-GO** and maturity level 2/5. It identifies missing fail-closed tenant enforcement, incomplete provider/payment finality, weak production observability, incomplete supply-chain release gates, and insufficient end-to-end accessibility proof. The completion roadmap prioritizes tenant isolation, durable telemetry, production-shaped verification, and WCAG 2.2 AA coverage.

The five candidates were selected because each maps to one of those verified blockers and is additional to the previously assessed Alibaba Open Code Review, Impeccable, last30days, claude-video, and Anthropic Skill Creator.

Tenant boundaries and database concurrency are not yet proven fail-closed	`docs/system-audit/STOQUIFY_WHOLE_SYSTEM_AUDIT_2026-08-03.md`; `docs/system-audit/STOQUIFY_COMPLETION_ROADMAP_2026-08-03.md`; `docs/architecture/decisions/0002-org-scoped-prisma-extension.md`	Supabase PostgreSQL guidance on RLS, schema, locking, connections, and query performance
CI lacks complete SAST/SCA/secrets/SBOM/container proof	`docs/system-audit/STOQUIFY_FINDINGS_REGISTER_2026-08-03.md`; whole-system audit release section	Trail of Bits maintainer/takeover-risk analysis as one advisory layer
Production telemetry and durable alert delivery are incomplete	whole-system audit observability findings; no Sentry/OpenTelemetry dependency or app integration found in `package.json` and application search	Sentry Next.js instrumentation and operational error context
Authenticated browser journeys need stronger reproducible evidence	completion roadmap golden-journey section; existing Playwright scripts and sanitized artifact controls	Anthropic's reconnaissance-first browser-testing method
WCAG 2.2 AA and performance proof remain incomplete	findings register and completion roadmap	Addy Osmani's cross-discipline quality checklist and thresholds

The graph report in `graphify-out/GRAPH_REPORT.md` reinforces that tenant defence, security foundations, enterprise error handling, `SystemMonitor`, and `ResilientDatabase` are cross-cutting nodes. That graph was generated earlier than the 2026-08-03 audit, so it was used only for navigation and architectural context, not as current completion evidence.

Method and Decision Rules

The assessment inspected the exact upstream `SKILL.md`, referenced scripts/resources, license, repository activity, and official documentation. It also checked Stoquify's stack, current scripts, existing browser tests, database architecture, release findings, and installed dependencies. None of the five exact candidate skills was found installed globally, project-locally, or in the repository at the time of review.

Each criterion was scored from 0 to 10. Weighted points equal `(score / 10) x weight`. Scores are decision aids, not certifications. A hard blocker overrides the numeric band.

Stoquify use-case and business value	20
Security, privacy, and data governance	15
Codex and Windows compatibility	15
Technical and architectural fit	10
Maintenance health	10
License and provenance clarity	10
Reliability and evidence quality	10
Installation and operational burden	5
Incremental value over existing capabilities	5

Verdict bands are 80-100 adopt, 65-79.9 controlled pilot, 50-64.9 selectively adapt/monitor, and below 50 reject.

1. The Database Guardian

Candidate

Skill: Supabase `supabase-postgres-best-practices` **Publisher:** Supabase **Primary sources:** repository, exact skill, RLS reference, MIT license, releases

What it does and how it operates

This is primarily an instruction-and-reference skill rather than an executable agent. It organizes PostgreSQL guidance into query performance, connection management, security/RLS, schema design, concurrency and locking, data-access patterns, monitoring, and advanced features. The selected skill does not require credentials or an external service merely to review code and migrations. That gives it a small execution surface and makes least-privilege, report-only use practical.

The RLS guidance includes both generic PostgreSQL session-context patterns and Supabase-specific `auth.uid()` examples. Stoquify uses Prisma with PostgreSQL/Neon and its own identity/organization model, so the principles fit but the Supabase Auth expressions do not transfer literally.

Stoquify value

This skill addresses the most consequential local risk. Stoquify's service-layer organization scoping is valuable, but the audit does not accept it as fail-closed proof for every path. The skill can improve reviews of:

1. tenant-scoped schema constraints and composite keys;
1. transaction isolation and concurrent balance/stock updates;
1. lock ordering, deadlock avoidance, and idempotent retry design;
1. connection-pool behavior in serverless/Neon deployment;
1. query plans and indexes for close, reconciliation, and operational dashboards;
1. an RLS pilot where justified by the roadmap.

It does not replace Prisma integration tests, migrations, database constraints, service-owned authorization, or security review. Its radical value is that it can turn a broad P0 risk into a repeatable database-review rubric with very little new runtime surface.

Risks and limitations

1. **Architecture mismatch:** copying Supabase Auth conventions into Better Auth/Prisma/Neon would create false confidence or broken policies.
1. **Split authorization:** careless RLS introduction can diverge from the service-layer tenant vocabulary and complicate pooling/session state.
1. **Generated advice:** recommendations are not proof until backed by migrations, cross-tenant negative tests, query plans, and concurrency tests.
1. **Version drift:** the repository is maintained and released, but Stoquify should still pin the reviewed commit and re-review updates.
1. **Scope:** it is PostgreSQL guidance, not an application authorization or accounting-correctness certification.

Grounded scenarios

1. **High value:** review a tenant-owned ledger migration and require a composite tenant key, a negative cross-tenant test, and a transaction plan before approval.
2. **Normal use:** examine a slow close dashboard query with `EXPLAIN (ANALYZE, BUFFERS)` and validate indexes against representative data.

3. **Failure case:** an agent copies `auth.uid()` into a Neon migration, assumes Supabase session semantics, and declares tenant isolation complete without verifying Prisma pool/session behavior.

Score

Criterion	Weight	Score (1–10)	Weighted score
Use-case and business value	20	10	20.0
Security/privacy/governance	15	9	13.5
Codex/Windows compatibility	15	9	13.5
Technical fit	10	9	9.0
Maintenance	10	9	9.0
License/provenance	10	10	10.0
Reliability/evidence	10	9	9.0
Operational burden	5	10	5.0
Incremental value	5	9	4.5
Total	**100**		**93.5**

Recommendation

Adopt now, governed and pinned. Use it first as a report-only reviewer for the P0 tenant-isolation work and database concurrency changes. Do not auto-apply SQL or migrations.

Pilot gate:

1. Pin an exact upstream commit and record its license and checksum.
1. Select one tenant-owned table and one concurrent financial or stock operation.
1. Require a principal database/security reviewer to translate Supabase-specific identity examples into Stoquify's organization context.
1. Pass cross-tenant negative tests, migration rollback, representative query-plan checks, and concurrency/idempotency tests.
1. Stop if the proposal introduces session-context leakage through pooled connections, duplicates authority logic inconsistently, or cannot be proven fail-closed.

---

2. The Web Quality Gate

Candidate

Skill: Addy Osmani `web-quality-audit` **Publisher:** Addy Osmani; community/unofficial repository **Primary sources:** repository, exact skill, MIT license, WCAG 2.2 Quick Reference, Lighthouse overview

What it does and how it operates

The skill is a compact Markdown checklist covering performance, accessibility, SEO, and general web best practices. It references Lighthouse and Core Web Vitals targets such as LCP below 2.5 seconds, INP below 200 milliseconds, and CLS below 0.1, and includes a WCAG-oriented review list. The selected skill contains no mandatory executable helper or credential requirement.

Its low operational burden is a strength, but its output is only as reliable as the measurements and human review attached to it. The repository describes itself as unofficial. It should be treated as an experienced review framework, not a standards body's certification instrument.

Stoquify value

Stoquify already has Playwright and UI smoke coverage, and Impeccable was previously considered as a design skill. This candidate adds a different layer: a cross-functional release checklist that links performance, accessibility, SEO/public surfaces, and browser best practices. Its best value is to standardize evidence expectations for the five authenticated EN/FR golden journeys in the completion roadmap.

It can require each journey to carry:

1. automated `axe` or equivalent findings with reviewed exceptions;
1. keyboard-only execution and focus-order evidence;
1. 200% zoom/reflow checks;
1. screen-reader review for critical workflows;
1. Lighthouse or browser performance traces on representative builds;
1. explicit mobile and low-bandwidth checks;
1. a retained artifact and accountable reviewer.

Risks and limitations

1. **Checklist theatre:** a completed checklist can be mistaken for tested accessibility or performance.
1. **False certification:** the skill cannot declare WCAG 2.2 AA conformance by itself.
1. **Coverage overlap:** some recommendations overlap existing Playwright smoke tests and the earlier Impeccable design candidate.
1. **Community maintenance:** provenance is clear and MIT-licensed, but the repository has a smaller maintenance surface than official platform repositories.
1. **Measurement context:** lab Lighthouse results do not prove real-user performance; authenticated and data-heavy routes require representative test conditions.

Grounded scenarios

1. **High value:** turn the payroll approval, reconciliation, close, inventory, and manager-action journeys into a WCAG/performance evidence matrix in both English and French.
2. **Normal use:** review a dashboard change for focus management, semantic landmarks, responsive reflow, LCP, INP, CLS, and console errors before release.
3. **Failure case:** a team checks every Markdown box using an unauthenticated landing page and labels the authenticated application WCAG-compliant.

Score

Use-case and business value	20	8	16.0
Security/privacy/governance	15	9	13.5
Codex/Windows compatibility	15	9	13.5
Technical fit	10	9	9.0
Maintenance	10	7	7.0
License/provenance	10	10	10.0
Reliability/evidence	10	6	6.0
Operational burden	5	10	5.0
Incremental value	5	7	3.5
Total	**100**		**83.5**

Recommendation

Adopt now as a governed checklist, not as certification. Convert its findings into Stoquify-native acceptance criteria and deterministic evidence. Keep the upstream content pinned and separate from generated project evidence.

Pilot gate:

1. Apply it to one authenticated EN/FR golden journey and one public page.
1. Require automated accessibility output, keyboard evidence, a manual screen-reader note, and repeatable performance measurements.
1. Store route, build, browser, locale, viewport, data fixture, and timestamp with the evidence.
1. Stop if the checklist produces untraceable subjective claims, noisy output without owners, or any claim of standards compliance without qualified review.

---

3. The Observability Engineer

Candidate

Skill: Sentry `sentry-nextjs-sdk` **Publisher:** Sentry **Primary sources:** repository, exact skill, official Next.js documentation

What it does and how it operates

The skill guides installation and configuration of `@sentry/nextjs` across browser, server, and edge runtimes, including Next.js App Router and current Next.js/Turbopack considerations. It proposes the Sentry wizard, manual initialization files, source-map upload credentials, error capture, tracing, and optional session replay. This is not merely a review skill: useful operation normally adds a runtime SDK and transmits telemetry to an external Sentry service.

The upstream examples materially affect the verdict. They set `sendDefaultPii: true`; the server example also sets `includeLocalVariables: true`. Other configuration documentation notes that PII transmission is normally off by default, but the skill's copy-ready examples turn it on. For Stoquify, that is a hard blocker to as-is adoption.

Stoquify value

The local audit identifies production observability as incomplete, while the dependency and code search found no current Sentry or OpenTelemetry integration. A carefully bounded Sentry pilot could improve:

1. provider webhook and settlement-job failure diagnosis;
1. server-action and route error correlation;
1. browser/server/edge release regressions;
1. alert routing and response ownership;
1. sanitized operational context for hard-to-reproduce failures.

This can materially reduce mean time to detect and resolve incidents. It cannot become the authoritative audit ledger, payment evidence store, or close-certification system.

Risks and limitations

1. **Hard privacy blocker:** `sendDefaultPii: true` is unacceptable as a default for financial, payroll, authentication, or tenant-sensitive routes.
1. **Local-variable exposure:** server local variables can contain secrets, tokens, request bodies, employee data, provider payloads, and financial evidence.
1. **Session replay:** replay can capture sensitive screens and user behavior unless disabled or rigorously masked and scoped.
1. **External transfer:** DSN-based telemetry, source maps, and attachments require data-processing, residency, retention, access, and deletion review.
1. **Credential risk:** source-map upload tokens must be CI-scoped, secret-managed, and excluded from logs and client bundles.
1. **Vendor dependence and cost:** useful retention, replay, tracing volume, and alerts create ongoing operational and commercial ownership.
1. **Windows friction:** the skill includes Unix-oriented discovery commands, although the underlying Next.js SDK is cross-platform and the commands can be translated.

Grounded scenarios

1. **High value:** a settlement reconciliation worker fails only in production; sanitized correlation tags identify provider, environment, release, and retry class without recording account data or payloads.
2. **Normal use:** an App Router server action throws after a deployment; the release and trace identify the affected route and regression while the authoritative audit record remains in Stoquify.
3. **Failure case:** copied defaults send payroll form fields, user identity, request data, local secrets, and replay footage to an external tenant.

Score

Criterion	Weight	Score (0–10)	Weighted score
Use-case and business value	20	10	20.0
Security/privacy/governance	15	3	4.5
Codex/Windows compatibility	15	8	12.0
Technical fit	10	10	10.0
Maintenance	10	8	8.0
License/provenance	10	9	9.0
Reliability/evidence	10	8	8.0
Operational burden	5	4	2.0
Incremental value	5	10	5.0
Total	**100**		**78.5**

Recommendation

Controlled, privacy-hardened pilot only. Do not install or configure the upstream examples unchanged. The numeric score would support a pilot, while the PII/local-variable defaults block direct adoption.

Mandatory pilot controls:

1. `sendDefaultPii: false` and `includeLocalVariables: false` in every runtime.
1. Disable session replay initially; do not enable it on finance, payroll, identity, billing, provider, evidence, or administrative routes.
1. Implement and test `beforeSend`/event scrubbing with synthetic secrets, tokens, account identifiers, employee fields, request bodies, headers, cookies, and URLs.
1. Use a staging-only Sentry project, least-privilege CI token, short retention, restricted roles, and an approved data-processing/residency decision.
1. Begin with one non-sensitive route or background job and a small error budget.
1. Promote only after zero forbidden fields appear in adversarial scrubbing tests and incident owners demonstrate alert/runbook handling.
1. Roll back by removing DSN/config and SDK initialization if data leakage, unacceptable bundle/runtime cost, noisy alerts, or unclear data ownership appears.

---

4. The Supply-Chain Sentinel

Candidate

Skill: Trail of Bits `supply-chain-risk-auditor` **Publisher:** Trail of Bits **Primary sources:** repository, exact skill, CC-BY-SA-4.0 license

What it does and how it operates

This skill uses repository and package metadata to flag dependencies with elevated maintenance or takeover risk. Its workflow considers maintainer count, staleness, popularity, suspiciously high-risk capabilities, disclosed vulnerabilities, and security-contact posture. It relies on shell-oriented investigation and the GitHub CLI, then writes a report such as `.supply-chain-risk-auditor/results.md`.

The upstream skill explicitly limits its scope: it is not active vulnerability scanning, runtime dependency analysis, or license-compliance verification. That candour improves trust, but it also means the skill cannot satisfy Stoquify's missing SCA/SBOM/container/security release gates alone.

Stoquify value

Stoquify has a large JavaScript dependency surface and a system-audit finding for incomplete supply-chain proof. Existing `npm audit`-style checks focus on known advisories; this skill adds a useful socio-technical signal: a dependency can be risky because it is abandoned, single-maintainer, newly transferred, or security-process-poor even when no CVE exists.

The best use is a quarterly or release-candidate risk review of direct production dependencies, especially authentication, database, payments, document parsing, file upload, browser automation, and build tooling. It should enrich, not replace, machine-readable SCA and SBOM outputs.

Risks and limitations

1. **Incomplete security coverage:** no active scanning, transitive-runtime proof, exploitability analysis, SBOM generation, or complete license analysis.
1. **Heuristic false positives/negatives:** maintainer and popularity signals are useful but cannot determine whether a package is safe.
1. **Shell/platform friction:** Bash-oriented instructions and `gh` assumptions require PowerShell/Codex adaptation and process restrictions on Windows.
1. **Network and metadata trust:** GitHub and registry metadata can be incomplete, rate-limited, renamed, or manipulated.
1. **Output path:** generated reports must use Stoquify's evidence locations and redaction policy rather than silently writing an unmanaged hidden file.
1. **License:** CC-BY-SA-4.0 is clear, but modified redistribution of skill text requires attribution/share-alike review; keep third-party content segregated and recorded.

Grounded scenarios

1. **High value:** identify a lightly maintained direct dependency on a critical authentication or payment path and trigger an owner, replacement analysis, or compensating controls before release.
2. **Normal use:** enrich the release dependency review with maintainer concentration, last-release activity, security policy, and known-advisory context.
3. **Failure case:** the team treats a clean heuristic report as proof of no vulnerabilities and omits SCA, SBOM, secret, CI-workflow, and container scans.

Score

Use-case and business value	20	8	16.0
Security/privacy/governance	15	8	12.0
Codex/Windows compatibility	15	6	9.0
Technical fit	10	8	8.0
Maintenance	10	8	8.0
License/provenance	10	7	7.0
Reliability/evidence	10	7	7.0
Operational burden	5	6	3.0
Incremental value	5	9	4.5
Total	**100**		**74.5**

Recommendation

Controlled advisory pilot. Translate the workflow to explicit read-only PowerShell/Codex commands, pin the source, and store output in Stoquify's release-evidence structure. Do not make it the release gate.

Pilot gate:

1. Audit only direct production dependencies first; no package changes.
1. Pair findings with a real vulnerability scanner, lockfile analysis, SBOM generation, secret scanning, CI workflow scanning, and container scanning where applicable.
1. Require evidence links and confidence for every finding; route decisions to named dependency owners.
1. Track confirmed material risks, false-positive rate, investigation time, and remediated/replaced dependencies.
1. Stop if the workflow produces unactionable popularity rankings, requires broad shell/network authority, or is used as a substitute for deterministic security tooling.

---

5. The Browser Certifier

Candidate

Skill: Anthropic `webapp-testing` **Publisher:** Anthropic **Primary sources:** repository, exact skill, skill-level Apache-2.0 license, server helper source

What it does and how it operates

The skill teaches a sensible browser-testing sequence: inspect the page, identify selectors and state, then perform actions and capture evidence. It uses Python's synchronous Playwright API and includes a `with_server.py` helper to launch one or more local servers before running a test command.

The helper invokes child processes with `shell=True` and captures stdout/stderr pipes without continuously draining them. On a chatty development server, undrained pipes can block. On Windows, terminating the parent shell may also leave a child process tree running. The skill asks the agent to treat the helper as a black box after viewing `--help`; that conflicts with Stoquify's requirement to review third-party execution code before trusting it.

Stoquify value

The reconnaissance-before-action pattern is good and can improve exploratory browser evidence. However, Stoquify already has a substantial JavaScript/TypeScript Playwright setup, authenticated storage-state handling, browser smoke scripts, and privacy-aware artifact sanitization. Adding a second Python Playwright harness would fragment selectors, fixtures, authentication setup, reporting, and process lifecycle.

The useful increment is therefore methodological:

1. capture current route/state before interaction;
1. prefer semantic roles and stable labels;
1. separate observation, action, assertion, and artifact capture;
1. attach reproducible context to failures;
1. sanitize screenshots, traces, videos, URLs, and logs.

Risks and limitations

1. **Duplicate test stack:** Python Playwright would compete with the existing TypeScript suite.
1. **Process risk:** `shell=True`, undrained pipes, and Windows process-tree behavior make the helper unsuitable as-is.
1. **Black-box conflict:** Stoquify should not execute an external helper it has been told not to inspect.
1. **Data leakage:** browser screenshots, traces, videos, storage state, and console/network logs can expose tenant, payroll, financial, or authentication data.
1. **Brittleness:** exploratory scripts can pass visually while missing authoritative business assertions.
1. **Low incremental value:** the repo already owns most of the underlying capability.

Grounded scenarios

1. **High value:** adapt the observation-first pattern to diagnose a failing authenticated reconciliation journey and attach sanitized state, locator, assertion, and trace evidence.
2. **Normal use:** inventory semantic roles and form labels before writing a stable TypeScript Playwright test for a new workflow.
3. **Failure case:** run the Python helper against the Next.js dev server, deadlock on captured output or leave child processes alive, and persist an unsanitized authenticated trace.

Score

Criterion	Weight	Score (1–10)	Weighted score
Use-case and business value	20	7	14.0
Security/privacy/governance	15	5	7.5
Codex/Windows compatibility	15	5	7.5
Technical fit	10	7	7.0
Maintenance	10	8	8.0
License/provenance	10	10	10.0
Reliability/evidence	10	6	6.0
Operational burden	5	4	2.0
Incremental value	5	3	1.5
Total	**100**		**63.5**

Recommendation

Selectively adapt; do not install the Python helper as-is. Add the reconnaissance-before-action method to Stoquify's existing TypeScript Playwright conventions and privacy controls.

Success gate:

1. Demonstrate that the adapted method reduces flaky selectors or triage time on two existing authenticated journeys.
1. Keep the current TypeScript fixtures, storage-state policy, artifact sanitization, timeouts, and teardown controls.
1. Require business-state assertions, not screenshots alone.
1. Stop if adaptation creates a parallel harness, weakens artifact redaction, or adds process-management complexity without measurable reliability improvement.

Cross-Skill Risk and Hard-Blocker Register

PII and local variables transmitted by copy-ready defaults	Sentry	Critical	Hard block to as-is adoption; force privacy-off defaults and adversarial scrubbing tests
Session replay captures sensitive application surfaces	Sentry	Critical	Disable for pilot; separate governance decision before any later use
Supabase identity examples copied into Neon/Prisma architecture	Supabase	High	Translate through Stoquify identity/tenant vocabulary; database and security review
Heuristic dependency report mistaken for complete security proof	Trail of Bits	High	Pair with SCA/SBOM/secrets/CI/container gates; label output advisory
Shell and process helper behaves poorly on Windows	Anthropic	High	Do not install helper; adapt method into existing TypeScript harness
Checklist mistaken for accessibility certification	Addy Osmani	High	Require automated and manual evidence plus qualified review
External content changes without review	All	Medium	Pin commit, record checksum/license, review diffs, maintain provenance manifest
Agent recommendations modify high-value business state	All	Critical	Report-only use; no finance/payroll/statutory/entitlement/provider write tools

Combined Operating Model

The five skills should feed Stoquify's existing evidence and release system, not create a separate authority plane.

``text Pinned external guidance | v Stoquify-specific wrapper and guardrails | v Read-only review or staging pilot | v Deterministic tests and retained artifacts | v Named human reviewer and existing release gate ``

Important interactions:

1 Supabase guidance can shape tenant-safe schemas and tests; Sentry must never record the tenant context or payload used to prove those boundaries.

1 Addy's checklist can define the browser-quality evidence; Anthropic's observation-first method can improve how that evidence is captured inside the existing TypeScript suite.

1 Trail of Bits can flag risky dependencies introduced by Sentry or browser tooling, but deterministic SCA/SBOM gates remain independent.

1 None of the five can certify accounting correctness, statutory readiness, WCAG conformance, security, or production release on its own.

Phased Recommendation

Phase 0 - Governance and provenance (days 1-3)

Owner: platform/security lead

Actions:

1. Record exact upstream commit, license, checksum, reviewed files, allowed tools, prohibited data, and update owner for every adopted or piloted skill.
1. Keep vendor content project-local or in an approved managed skill registry; do not silently replace Stoquify instructions.
1. Require report-only operation and prohibit production credentials and business writes.

Gate: a reviewer can reproduce the source and explain every executable path. **Stop:** unclear license, unreviewed script, broad shell authority, or mandatory sensitive-data transfer.

Phase 1 - Database and web-quality adoption (weeks 1-2)

Owners: principal database engineer; accessibility/QA lead

Actions:

1. Run the PostgreSQL skill in report-only mode against one P0 tenant-isolation slice and one concurrent business operation.
1. Convert the web-quality checklist into evidence requirements for one authenticated EN/FR golden journey.

Metrics:

1. cross-tenant negative tests pass on every reviewed path;
1. query/concurrency claims carry deterministic evidence;
1. the golden journey has automated accessibility output, keyboard and screen-reader notes, repeatable performance data, and sanitized artifacts;
1. zero unsupported certification claims.

Phase 2 - Supply-chain advisory pilot (week 2)

Owner: application security/release engineering

Actions:

1. Review direct production dependencies and compare findings with SCA, lockfile, SBOM, secret, CI-workflow, and container evidence.
1. Assign owners only to substantiated risks.

Metrics: confirmed material findings, false-positive rate, investigation time, owner coverage, and closure rate. **Stop:** the output cannot be reproduced, is dominated by popularity noise, or displaces deterministic gates.

Phase 3 - Privacy-hardened observability pilot (weeks 3-4)

Owners: SRE/operations plus privacy/security approver

Actions:

1. Instrument one non-sensitive staging route or worker with PII off, local variables off, replay off, aggressive scrubbing, short retention, and least-privilege access.
1. Seed synthetic secrets and sensitive fields to prove they are removed before transmission.

Metrics: zero forbidden fields across adversarial tests; actionable alert rate; alert acknowledgement; diagnostic time improvement; runtime and bundle overhead. **Stop:** any sensitive-field leakage, unclear data residency/ownership, unacceptable cost/overhead, or noisy unowned alerts.

Phase 4 - Browser-method adaptation (only if needed)

Owner: frontend QA

Actions:

1. Add observation-first guidance to existing TypeScript Playwright conventions; do not add the Python helper.
1. Measure flake rate and triage time on two authenticated journeys.

Gate: measurable improvement with no parallel harness and no artifact-policy regression. **Stop:** no measurable improvement after two journeys.

What These Skills Add Beyond the Existing Estate

Stoquify already has many domain-specific execution and assurance skills. The earlier five external candidates focused on code review, visual design, recent research, video understanding, and skill authoring. This new set is more infrastructural:

1. Supabase adds deep PostgreSQL review logic at the tenant/concurrency boundary.
1. Trail of Bits adds maintainer and takeover risk that conventional CVE scanning misses.
1. Sentry adds production error context and alerting, subject to unusually strict privacy controls.
1. Anthropic contributes a browser investigation method, not a needed new runtime.
1. Addy Osmani adds a unified performance/accessibility/quality review frame.

Their value is highest when they strengthen Stoquify's existing evidence plane and lowest when they introduce a parallel runtime, authority vocabulary, test harness, or certification claim.

Final Go/No-Go Decisions

Supabase PostgreSQL	No automatic installation	**GO** for pinned, report-only adoption	Review one P0 tenant slice and one concurrency path
Addy web quality	No automatic installation	**GO** as a checklist backed by tools	Certify evidence collection, not WCAG status, on one EN/FR journey
Sentry Next.js	**No**	**CONDITIONAL GO** for staging-only privacy pilot	Write and approve a deny-by-default telemetry data contract before adding SDK code
Trail supply-chain auditor	No automatic installation	**CONDITIONAL GO** as advisory pilot	Compare direct-dependency findings with deterministic release scans
Anthropic webapp testing	**No helper installation**	**NO-GO as a new harness; GO for method adaptation**	Add observation-first guidance to one existing TypeScript Playwright workflow

Sources Reviewed

Local Stoquify evidence

1. `package.json`
1. `docs/system-audit/STOQUIFY_WHOLE_SYSTEM_AUDIT_2026-08-03.md`
1. `docs/system-audit/STOQUIFY_COMPLETION_ROADMAP_2026-08-03.md`
1. `docs/system-audit/STOQUIFY_FINDINGS_REGISTER_2026-08-03.md`
1. `docs/architecture/decisions/0002-org-scoped-prisma-extension.md`
1. `docs/stoquify-skills-agents/STOQUIFY_TOP_12_ADDITIVE_AGENTS_AND_SKILLS_RESEARCH_REPORT_2026-08-02.md`
1. `graphify-out/GRAPH_REPORT.md`

1. Playwright, release-gate, privacy-artifact, package, dependency, observability, and database-access files found through repository search

External primary sources

1. Supabase: repository, skill, RLS guidance, license, releases
1. Trail of Bits: repository, skill, license
1. Sentry: repository, skill, official Next.js SDK documentation
1. Anthropic: repository, skill, license, helper source
1. Addy Osmani: repository, skill, license, WCAG 2.2, Lighthouse

Limitations and Confidence

This is a static, read-only assessment dated 2026-08-05. No candidate was installed, executed, authenticated, or tested against production-like Stoquify data. Repository activity, skill contents, service pricing, and platform behavior can change after this date. Upstream popularity was not treated as proof of security or usefulness.

Confidence is **high** in the architecture, privacy, overlap, and licensing conclusions because they are grounded in local files and exact upstream skill/script content. Confidence is **medium** in maintenance longevity and operational effort until commit-pinned pilots run on Stoquify's Windows/Codex environment.

Final Recommendation

Start with a commit-pinned, report-only application of Supabase's PostgreSQL skill to one P0 tenant-isolation slice. It is the best combination of business impact, architectural fit, low execution risk, and evidence potential. In parallel, adopt the web-quality skill only as a checklist whose claims must be supported by real browser and accessibility artifacts.

Do not configure Sentry until Stoquify has approved a deny-by-default telemetry data contract. Do not treat the Trail of Bits skill as a complete supply-chain gate. Do not introduce Anthropic's Python browser helper into the existing TypeScript Playwright estate.

That sequence improves the database boundary first, then the quality evidence, then the operational signal - without weakening Stoquify's service-owned authority or exposing sensitive business data.